SECJ1023 Section 02

Programming Technique II

Project Deliverable 2 Report

Group Name: False 9

Group members:

Name	No Matric
Muhammad Najmi Shahmi Bin Mohd Latpi	A25CS0279
Nasaaie Bin Noriskamar	A25CS0118
Akmal Rafique Bin Ahmad Raphaie	A25CS0181
Muhammad Fairuz Bin Herman	A25CS0267

Table Of Content

1. Problem Analysis

Video link : https://youtu.be/6S3rpopuHMY?si=Bnlhos-H6yAsSS1b

Based on the brainstorming session and storyboard that have been created, we found that this game concept uses an Object-Oriented Programming approach. The purpose of this approach is so that software architecture can be managed systematically. The main challenge that we have identified is in terms of dynamic entities, the insects that move simultaneously in large quantities towards a specific boundary (Picnic Zone). Interaction between users through precise mouse clicks will be detected by this game system, which will give points or assign penalties.

(a) Identify objects and classes

Objects

The initial step in the system design phase involves the identification of the existing physical object in the computer memory during the runtime.

Specific Insects	Entity that is being displayed on the screen (Eg. an ant at coordinate 100,200)
Group Of Insects	A collection that holds all the active insects entity
Game session	An invisible manager who tracks the the game status such as "Playing" or "Game Over"
The User Interface	A visual representation of remaining heart and total score of the player

Classes

For development purposes, these objects will be summarized into the following Classes, complete with each significant attribute and methods. In order to ensure the principle of encapsulation, attributes are set as private or protected, while data manipulation is carried out through public methods.

1. **Insect (Base Class)**

Description : The base framework of all insect entities in the game

Attribute	- x : int - y : int - health : int - points : int
Method	+ move() : void + draw() : void + onClicked() : int + isHit(mx: int, my: int) : bool

2. **Ant (Derived Class)**

Description : The standard enemy

Attribute	-
Method	+ Ant(startX:int,startY:int)

3. **Beetle (Derived Class)**

Description : The tank categorical enemy that requires multiple attack

Attribute	-
Method	+ Beetle(startX: int, startY: int) + onClicked() : int

4. **Bee (Derived Class)**

Description : A trap entity that leads to a penalty.

Attribute	- angle : float
Method	+ Bee(sX: int, startY: int) + move() : void

5. **Game (Controller Class)**

Description : Manage the main game cycle (game loop), user input, and system state transitions

Attribute	- score : int - lives : int - swarm : vector<Insect*> - ui : UI
Method	+ spawnInsect() : void + checkCollisions() : void + updateScreen() : void

6. **Scoreboard (View Class)**

Description : Manage the Heads-Up Display (HUD) and menu

Attribute	-
Method	+ drawHUD(score: int, lives: int) : void, + drawGameOverScreen(finalScore: int) : void

(b) Identify and justification of class relationship

The next step after the identification of object and classes in system design phase involves the identification and justification of class relationship to ensure the code is modular and adheres to OOP principles

Inheritance Relationship

Class Involved	Ant, Beetle, and Bee class inherit from the base class which is Insect.
Justification	Share the same foundational properties such as coordinates, move downwards toward the Picnic Zone, and they all have hitboxes that respond to the mouse cursor. This can avoid code duplication. It also allows us to utilize Polymorphism. The Game class can loop through an array of Insect pointers and call move() or onClicked() methods. The program will automatically execute the correct specific behavior such as the Bee's zig-zag movement or the Beetle's armor logic.

Association Relationship

(i)

Class Involved	Game class has a relationship with the Insect class
Justification	Game object maintains a collection of Insect object which is the swarm attributes. Insects are continuously spawned and destroyed which means they get squashed or crossing the Picnic Zone while the Game object persists. They are collected by the game, but they are not permanent structural components of the game itself.

(ii)

Class Involved	Game class has a relationship with the Scoreboard class
Justification	The scoreboard exists to display the data which is 3 hears and the score of the users for a specific game session. If the user clicks “Quit Game” and the Game object is destroyed, the Scoreboard is destroyed alongside it .

2.0 UML Class Diagram